



Linker Hand L20

Product Manual

1. Product Overview

Product Overview

The Linker Hand L20 is a high-performance dexterous hand with 20 degrees of freedom. It can simulate the natural grasping method similar to that of a human hand, enabling more diverse and precise operations. Adopting the linkage transmission method and being driven by self-developed motors, it can achieve precise control and excellent motion performance.

Equipped with a variety of sensors, including force sensors, vision sensors, and tactile sensors, it can realize multi-modal perception, endowing it with stronger environmental adaptability and intelligent interaction capabilities. It is compatible with ROS and QT applications, provides ROS

plugins, and supports users' secondary development, thus expanding more application possibilities.

With its high degree of freedom, multi-modal perception ability, and force-position hybrid algorithm, the Linker Hand L20 can be widely applied in scenarios such as education and scientific research, industrial automation, household assistance, and health care and nursing.

Main Features

1. High Degree of Freedom

- Each finger independently has 4 degrees of freedom, meeting the needs of complex and precise operations.

2. Multi-sensors

- Equipped with an advanced multi-sensor system, including cameras, electronic skin, etc., it constructs an all-round visual perception mode, realizes precise environmental perception and interaction ability, and adapts to a variety of application scenarios.

3. Cloud-based intelligent

- With innovative cloud-based intelligent technology, users do not need to write code. They can quickly deploy through the skill library cloud service, achieve efficient customized operations, and reduce the difficulty of use.

4. Data Collection

- With the self-developed high-efficiency data collection capability, a data farm can be established.

5. Sturdy and Durable

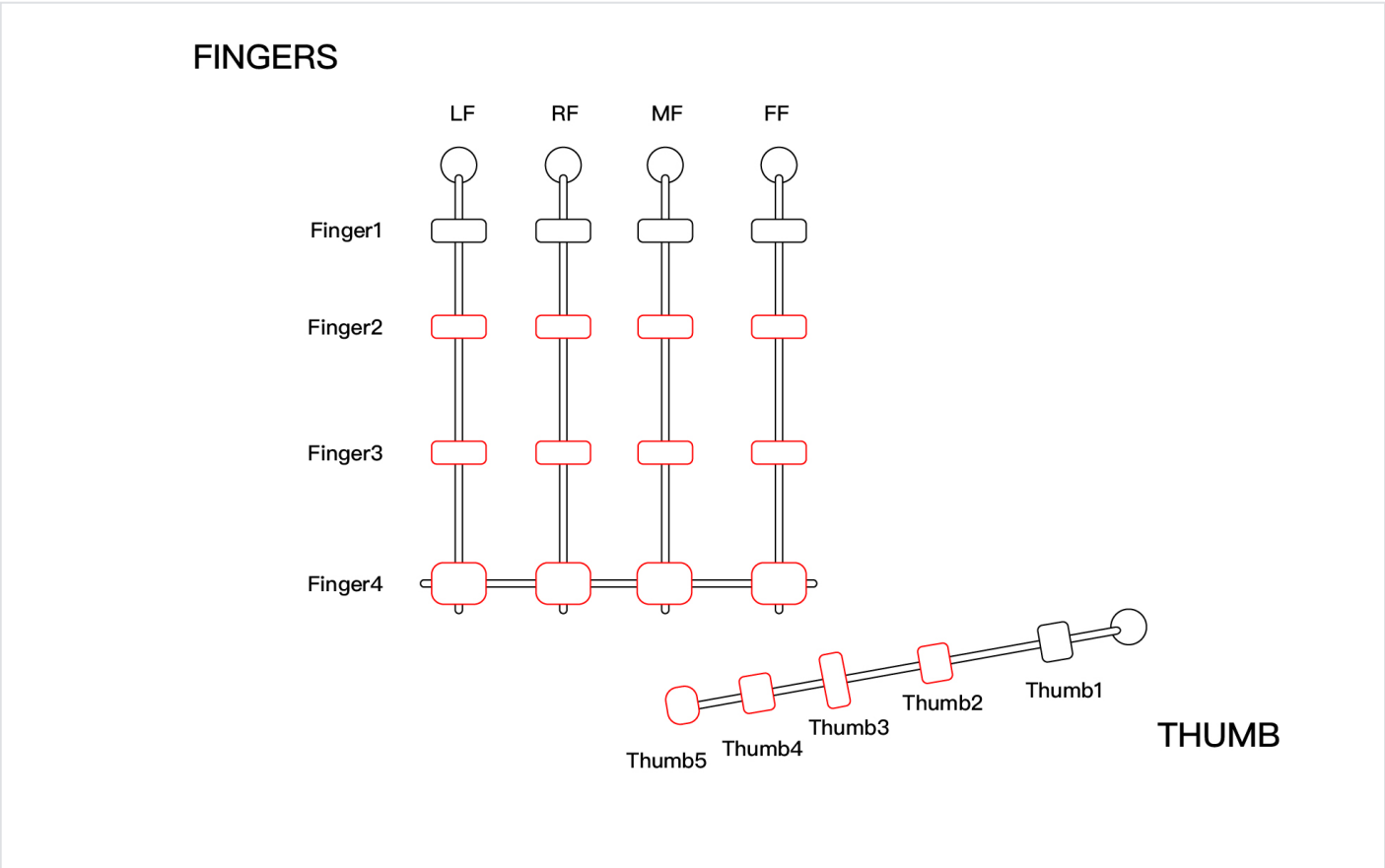
- It is impact-resistant and can withstand drops, making it suitable for various training scenarios of embodied intelligence.

2. Product Display



3. Example of Degrees of Freedom

L20 (16 ctive degrees of freedom+5 passive degrees of freedom)

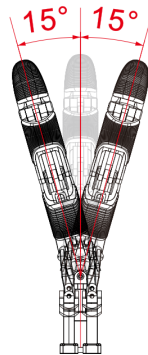


Joints	Min (°)	Max (°)	Note
LF2、RF2、MF2、FF2、THUMB2 (Bending)	0	LF3、RF3、MF3、FF3—>80 THUMB3—>74	
LF3、RF3、MF3、FF3、THUMB3 (Bending)	0	LF3、RF3、MF3、FF3—>87 THUMB3—>50	
LF4、RF4、MF4、FF4、THUMB4 (Side swing)	0	LF3、RF3、MF3、FF3—>±30 THUMB3—>95	
THUMB5(Rotation)	0	82	

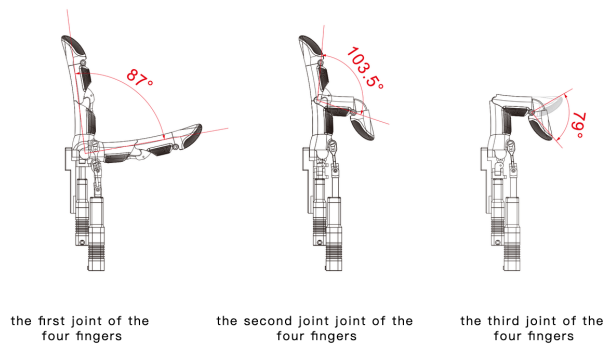
4. Motion Space Range

Four-finger Structure

For the four-finger structure of the Linker Hand L20 dexterous hand, the second joint and the first joint of the four fingers can be manually adjusted at an angle according to the shape of the object to be grasped. The second joint and the third joint of the four fingers achieve coupled linkage through the connection of linkages.

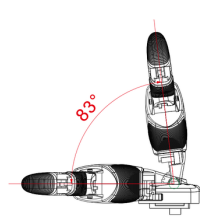


Four fingers side swing angle schematic diagram

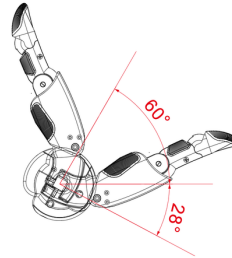


Thumb Structure

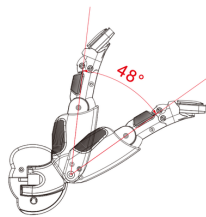
For the thumb structure of the Linker Hand L20 dexterous hand, the third joint and the second joint of the thumb achieve coupled linkage through the connection of linkages, and the first joint of the thumb is independently driven by a rotating motor.



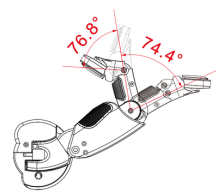
Side swing thumb



Rotating thumb



First joint of thumb



Second and third joints of thumb

5. Scene Demonstration





6. Sensor System

Tactile Sensing (Standard Configuration)

The Linker Hand L20 is equipped with fingertip sensors that utilize high-sensitivity sensing technology. It can predict and sense the presence and distance of objects. When in contact with an object, it can accurately capture three - dimensional forces and identify changes in surface texture and temperature.

Fingertip Tactile Sensors

Technical parameters		
sampling frequency		$\geq 50\text{Hz}$
		0-20N
force application	measurement accuracy	0.1N
	measurement resolution	0.5%FS
	measurement precision	2%FS
	measurement accuracy	0.25N
	directional resolution	45°

Miniature Three-dimensional Force Sensors Inside the Fingers

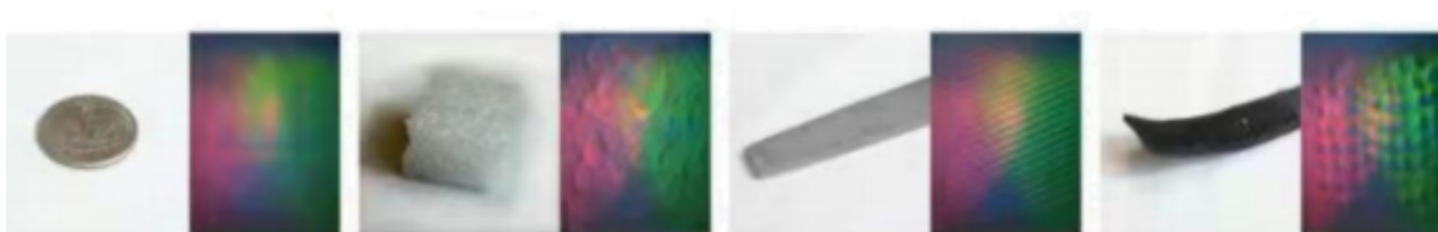
Technical parameters	
Measurement range	0-50N
maximum force endured	200N
measurement resolution	0 . 2N
fastest measurement rate	1KHz

Vision (Optional Configuration)

The Linker Hand dexterous hand adopts the design method of high-sensitivity fingertip cameras + palm cameras + wrist cameras, enabling multi-visual fusion perception. The minimum remote operation system is equipped with a depth camera on the arm.

Visual-tactile Perception (Optional Configuration)

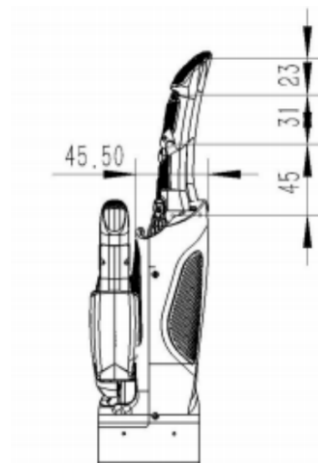
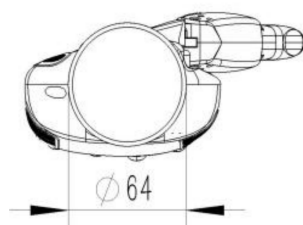
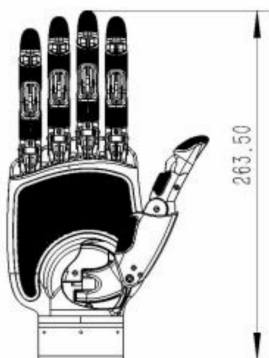
The Linker Hand dexterous hand has a visual-tactile perception mode, which essentially combines vision with a large deep learning model. The principle of this technical solution is to use a high-precision camera to capture the deformation of variable flexible materials. When subjected to force, the shape of the grid deforms, and our miniature binocular cameras record this deformation. Then, based on the well-trained large deep learning model, the depth information and movement trend of the object are mapped out.

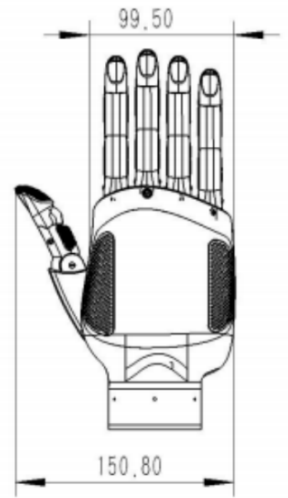


7. Product Parameters

Product	Linker Hand L20
Degrees of Freedom	18
Number of Joints	16+2active. 5passive
Transmission Mode	Linkage drive
Drive Mode	Self-developed joint module
Control Interface	CAN/RS485
Weight	1000g
Maximum Load	5kg
Operating Voltage	DC24V±10%
Static Current	0.2A
Average Current during No-load Movement	1A
Maximum Current	3A
Repeat Positioning Accuracy	±0.20mm
Maximum Grip Force of the Thumb	15N
Maximum Grip Force of the Four Fingers	10N
Maximum Lateral Rotation Range of the Thumb	1.54rad (88.3°)
Bending Angle of the Four Fingers	1.57rad (90°)
Side Swing Speed of the Thumb	3.28rad/s(188.17°/s)
Bending Speed of the Four Fingers	1.99rad/s (114.29°/s)
Bending Speed of the Thumb	1.07rad/s (62.26°/s)

8. Appearance Dimensions





9. Communication Methods

All versions of the dexterous hand support the use of the CAN bus debugging port or EtherCAT. EtherCAT (Ethernet for Control Automation Technology) is a fieldbus based on 100Mbps Ethernet. It is currently used in many systems, and the latest version of the Linker Hand can be well compatible with both the EtherCAT and ROS systems. Using EtherCAT or CAN in combination with ROS requires a multi-core PC with good performance or our AI-Box device, as well as a standard Ethernet port. The EtherCAT protocol used by the Linker Hand mainly relies on the upper host computer or AI-Box to complete this work.

Supported Functions

- Enable and disable position control
- Change the PID values for torque control
- Perform restrictive operations, such as cutting off force, current, temperature, etc.
- Reset the motor
- Adjust the data transmission rate of the motor and tactile sensors
- Track errors and status indicator lights in the components
- Download the latest firmware to the motor module
- Download the latest Skill function module to the hand controller
- Obtain data from the vision sensor

Control Strategy

Under the default configuration, EtherCAT can rely on the host computer or AI-Box to implement the position control strategy. More complex control algorithms can be used, which can integrate the information of joint and tactile sensors, and even achieve the integration of visual signals through ROS. The torque loop inside the motor unit closes at a frequency of 5kHz. The PID

settings of this loop can be changed in real time. If different control strategies are required, you can purchase existing control strategies from the Skill Store cloud service and use them directly without programming. It also supports downloading new firmware to the motor.

Microcontroller

The Linker Hand uses a self-developed microcontroller for the embedding of the entire robot system. All microcontrollers are connected to the internal CAN bus and can be accessed through the EtherCAT interface.

10. Code Examples

Examples of the Linker Hand ROS SDK program

examples

- [0000-linker_hand_pybullet](#) (PyBullet Simulation example)
 - [0001-get_linker_hand_state](#) (Obtain the current status of the Linker hand)
 - [0002-gui_control](#)(control it through the graphical interface)
 - [0003-get_linker_hand_force](#) (obtain the data of the Linker hand force sensor)
 - [0004-get_linker_hand_speed](#) (Obtain the current speed of the Linker hand force)
 - [0005-get_linker_hand_current](#) (Obtain the current current of the Linker hand force)
-
- [0101-lipcontroller](#) (The tactile sensor cooperates with the Linker hand to perform pinching actions)
 - [0102-gesture-Show-OK](#) (Use Python to control the hand to make an "OK" gesture)
 - [0103-gesture-Show-Surround-Index-Finger](#) (Use Python to control the hand to rotate the index finger)
 - [0104-gesture-Show-Wave](#) (Use Python to control the hand to make a wave motion)
 - [0105-gesture-Show-Ye](#) (Use Python to control the hand to perform a set of complex demonstration actions)
-
- [1001-human-dex](#) (Use the Linker Hand for imitation learning training and achieve autonomous object grasping)
 - [1002-linker_unidexgrasp](#) (The Unidexgrasp dexterous hand grasping algorithm based on the Linker Hand)

Instructions for the Linker Hand Configuration File

Whether it is the real LinkerHand or the simulated one, the parameter file needs to be configured first. Modify the corresponding configuration parameters according to actual needs.

(1) Modify the configuration file and configure it for the physical LinkerHand or the simulation environment.

```
1 $ cd Linker_Hand_SDK_ROS/src/linker_hand_sdk/linker_hand_sdk_ros/config
2 $ sudo vim setting.yaml
```

Since the graphical interface can only control one LinkerHand individually, corresponding configurations need to be made in the configuration file to match the physical LinkerHand.

Linker Hand Example 100

The LinkerHand Example 100 provides rich example cases and source code, fully demonstrating the functions of the LinkerHand.

- Preparation
Start the SDK

```
1 #Open a new terminal Start ros
2 $ roscore
```

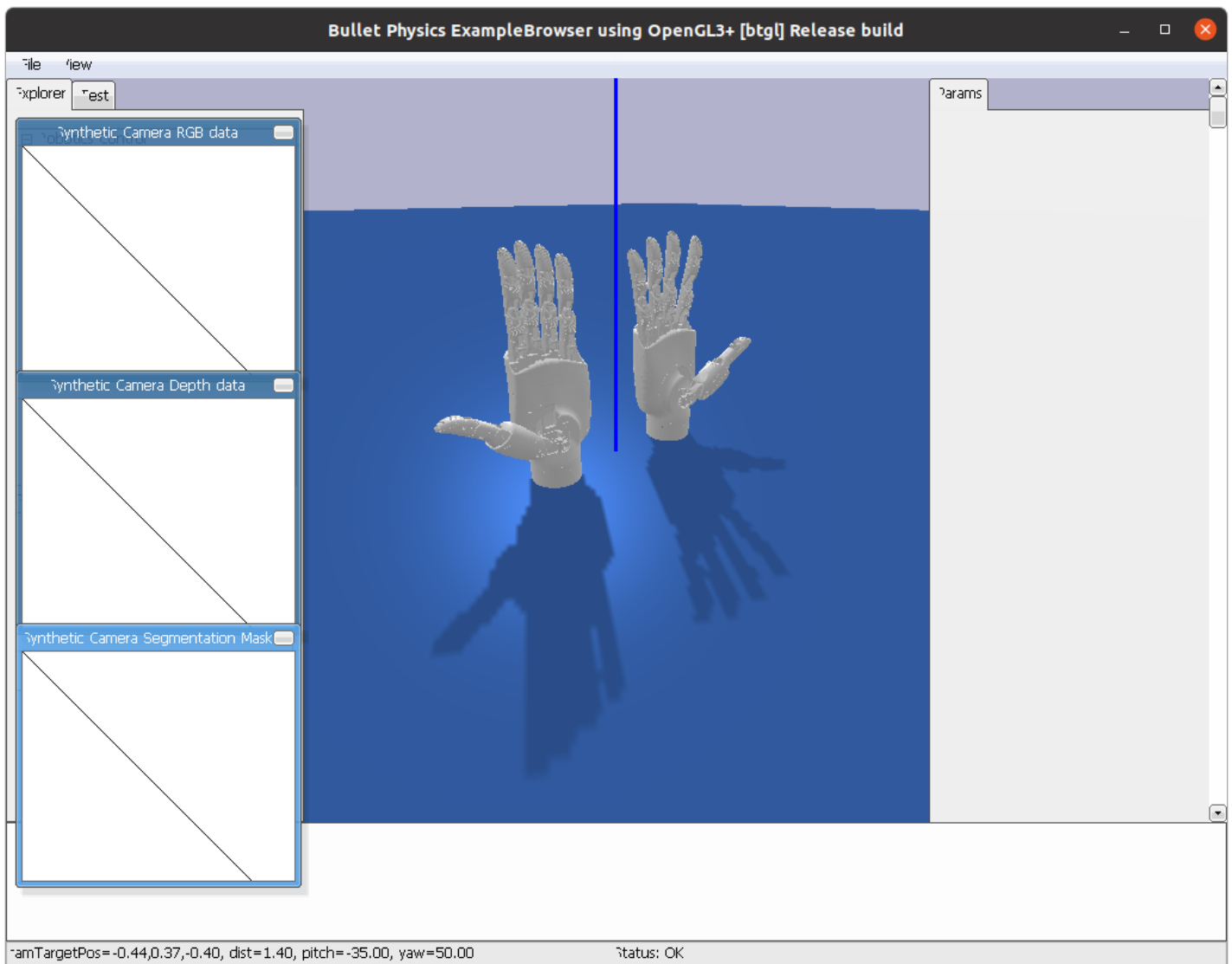
Open a new terminal and start ROS SDK

```
1 $ cd Linker_Hand_SDK_ROS/
2 $ source ./devel/setup.bash
3 $ roslaunch linker_hand_sdk_ros linker_hand.launch
```

0000-PyBullet Simulation Example

Open a new terminal. The SDK can control the LinkerHand in the simulator.

```
1 $ cd Linker_Hand_SDK_ROS/
2 $ source ./devel/setup.bash
3 $ rosrun linker_hand_pybullet linker_hand_pybullet.py
```



0001-Obtain the Current Status of the Linker Hand . The status values include range values and radian values

Open a new terminal

```
1 $ cd Linker_Hand_SDK_ROS/
2 $ source ./devel/setup.bash
3 _If the loop parameter is True, the terminal will print the current status values of the LinkerHand dexterous hand in a loop. If it is False, the terminal will only print the current status values of the LinkerHand dexterous hand once
4 $ rosrun L20_get_linker_hand_state L20_get_linker_hand_state.py _loop:=True
```

0002-Graphical Interface Control

The graphical interface control allows you to control the independent movement of each joint of the LinkerHand L10 and L20 through sliders. You can also add buttons to record the values of all current sliders and save the current movement status of each joint of the LinkerHand. Replay actions through functional buttons.

Controlling LinkerHand using gui_control:

To control the dexterous hand through the gui_control interface, you need to start linker_hand_sdk_ros and operate the LinkerHand in the form of a topic.

```
1  Open a new terminal Start ros
2  $ roscore
```

Open a new terminal start ROS SDK

```
1  $ cd Linker_Hand_SDK_ROS/
2  $ source ./devel/setup.bash
3  $ roslaunch linker_hand_sdk_ros linker_hand.launch
```

After a successful startup, there will be prompt information such as the SDK version, CAN interface status, dexterous hand configuration information, and the current joint speed of the Linker hand

Open a new terminal to start the gui control.

Copy the bash code

```
1  $ cd Linker_Hand_SDK_ROS/
2  $ source ./devel/setup.bash
3  $ rosrn gui_control gui_control.py
```

After starting, the UI interface will pop up. You can control the movement of the corresponding LinkerHand joints through the sliders. And you can save the current slider data by adding buttons on the right side for reuse

0003-Obtain the data of the force sensor of the Linker Hand

Open a new terminal

```
1  $ cd Linker_Hand_SDK_ROS/
2  $ source ./devel/setup.bash
3  _If the oop parameter is True, the terminal will print the current status
   values of the LinkerHand dexterous hand in a loop. If it is False, the
   terminal will only print the current status values of the LinkerHand
   dexterous hand once
4  $ rosrn get_linker_hand_force get_linker_hand_force.py _loop:=False
```

```
5 #2025-01-15 15:43:16 no data for the left hand#2025-01-15 15:43:16 The
normal force of the five fingers of the right hand: [0.0, 0.0, 0.0, 0.0,
0.0]#2025-01-15 15:43:16 The tangential force of the five fingers of the
right hand: [0.0, 0.0, 0.0, 0.0, 0.0]#2025-01-15 15:43:16 The direction of
the tangential force of the five fingers of the right hand: [255.0, 255.0,
255.0, 255.0, 255.0]#2025-01-15 15:43:16 The proximity sensing of the five
fingers of the right hand: [0.0, 0.0, 0.0, 0.0, 0.0]
```

0004-Obtain the current speed of the force of the Linker Hand

Open a new terminal

```
1 $ cd Linker_Hand_SDK_ROS/
2 $ source ./devel/setup.bash
3 _If the oop parameter is True, the terminal will print the current status
values of the LinkerHand dexterous hand in a loop. If it is False, the
terminal will only print the current status values of the LinkerHand
dexterous hand once
4 $ rosrun get_linker_hand_speed get_linker_hand_speed.py _loop:=False
5 #2025-01-15 15:57:17 no data for the left hand#2025-01-15 15:57:17 The
current of the five fingers of the right hand is: [180, 250, 250, 250, 250]
```

0005 - Obtain the current current of the force of the Linker Hand

Open a new terminal

```
1 $ cd Linker_Hand_SDK_ROS/
2 $ source ./devel/setup.bash
3 _If the oop parameter is True, the terminal will print the current status
values of the LinkerHand dexterous hand in a loop. If it is False, the
terminal will only print the current status values of the LinkerHand
dexterous hand once
4 $ rosrun get_linker_hand_current get_linker_hand_current.py _loop:=False
5 #2025-01-15 16:25:29 no data for the left hand#2025-01-15 16:25:29 The
current of the five fingers of the right hand is: [42, 42, 42, 42, 42]
```

0101-Pinch operation with the tactile sensor in cooperation with the Linker Hand

To use this example, you need to start linker_hand_sdk_ros

```
1  Open a new terminal Start ros
2  $ roscore
```

Open a new terminal start ROS SDK

```
1  $ cd Linker_Hand_SDK_ROS/
2  $ source ./devel/setup.bash
3  $ roslaunch linker_hand_sdk_ros linker_hand.launch
```

After a successful startup, there will be prompt information such as the SDK version, the status of the CAN interface, the configuration information of the dexterous hand, and the current joint speed of the dexterous hand.

Open a new terminal to use the demonstration example

```
1  python ./<Your file path>/lipcontroller.py
```

- If the terminal prints out "**Start the demonstration**", it means the normal operation. At this time, if the hand settings are correct, the index finger and middle finger should start to perform the pinching action. It will stop when pinching an object, and will continue to try to pinch after the object is taken away until it pinches an object or reaches the limit. The limit state is shown in the following figure



- [lipcontroller.py](#) This is a demonstration demo developed based on version 7. When applying it to the demonstrations of other versions, you need to adjust the opposing posture of the thumb and index finger. Otherwise, the action of "**pinching the index finger and thumb together**" cannot be achieved.

0102- Use Python to control the hand to make an "OK" gesture

To use this example, you need to start linker_hand_sdk_ros

Copy the bash code

```
1  Open a new terminal  Start ros
2  $ roscore
```

Open a new terminal start ROS SDK

```
1  $ cd Linker_Hand_SDK_ROS/
2  $ source ./devel/setup.bash
3  $ roslaunch linker_hand_sdk_ros linker_hand.launch
```

After a successful startup, there will be prompt information such as the SDK version, the status of the CAN interface, the configuration information of the Linker hand, and the current joint speed of the Linker hand.

```
1  python ./<Your file path>/gesture-Show-OK.py
2  #After starting, the terminal will print "Testing", and at this time, the
   hand will start to make an "OK" gesture, and the middle finger, ring finger,
   and little finger will bend and straighten
```

0103- Use Python to control the hand to make the index finger rotate

To use this example, you need to start linker_hand_sdk_ros

```
1  Open a new terminal  Start ros
2  $ roscore
```

Open a new terminal start ROS SDK

```
1  $ cd Linker_Hand_SDK_ROS/
```

```
2 $ source ./devel/setup.bash
3 $ roslaunch linker_hand_sdk_ros linker_hand.launch
```

After a successful startup, there will be prompt information such as the SDK version, the status of the CAN interface, the configuration information of the Linker hand, and the current joint speed of the Linker hand.

Open a new terminal to use the demonstration example

```
1 python ./<Your file path>/gesture-Show-Surround-Index-Finger.py
2 #After starting, the terminal will print "Testing", and at this time, the
  hand will start to clench into a fist and extend the index finger, and the
  index finger will keep repeating the rotation
```

0104- Use Python to control the hand to make wave motion

To use this example, you need to start linker_hand_sdk_ros

Copy the bash code

```
1 Open a new terminal Start ros
2 $ roscore
```

Open a new terminal start ROS SDK

```
1 $ cd Linker_Hand_SDK_ROS/
2 $ source ./devel/setup.bash
3 $ roslaunch linker_hand_sdk_ros linker_hand.launch
```

After a successful startup, there will be prompt information such as the SDK version, the status of the CAN interface, the configuration information of the Linker hand, and the current joint speed of the Linker hand.

Open a new terminal to use the demonstration example

Copy the bash code

```
1 python ./<Your file path>/gesture-Show-Wave.py
2 #After starting, the terminal will print "Testing", and at this time, the
  thumb of the hand will stretch outward and remain still, while the other four
  fingers will start to make wave motions
```

0105- Use Python to control the hand to perform a set of complex demonstration actions

To use this example, you need to start linker_hand_sdk_ros

```
1  Open a new terminal Start ros
2  $ roscore
```

Open a new terminal start ROS SDK

```
1  $ cd Linker_Hand_SDK_ROS/
2  $ source ./devel/setup.bash
3  $ roslaunch linker_hand_sdk_ros linker_hand.launch
```

After a successful startup, there will be prompt information such as the SDK version, the status of the CAN interface, the configuration information of the Linker hand, and the current joint speed of the Linker hand.

Open a new terminal to use the demonstration example

Copy the bash code

```
1  python ./<Your file path>/gesture-Show-Ye.py
2  #After starting, the terminal will print "Testing", and at this time, the
    hand will start to perform a set of complex movements to demonstrate the
    flexibility of the hand
```

-This example is a demonstration demo developed based on version 7. When applying it to the demonstrations of other versions, you need to adjust the opposing posture of the thumb and index finger. Otherwise, the action of "**pinching or opposing the index finger and thumb together**" cannot be achieved.

1001- Use LinkerHand for imitation learning training

To use this example, you need to use the ROS Noetic system on Ubuntu 20.04. The hardware is the LinkerRobot humanoid robot. You can also use other robotic arms or robots for imitation learning training, as long as you modify the corresponding data topics.

[For detailed usage instructions, please refer to thehuman-dex README.md](#)

1、Configuration environment

```
1  cd human-dex
2  conda create -n human-dex python=3.8.10
3  conda activate human-dex
4  pip install torchvision
5  pip install torch
6  pip install -r requirements.txt
```

2、install

```
1  mkdir -p your_ws/src
2  cd your_ws/src
3  git clone https://github.com/linkerbotai/human-dex.git
4  cd ..
5  catkin_make
6  source ./devel/setup.bash
```

3、run

```
1  collect data
2  roslaunch record_hdf5 record_hdf5.launch
3  open a new terminal to send the data collection command
4  rostopic pub /record_hdf5 std_msgs/String "data:
    '{"method\":"start\","type\":"humanplus\"}'"
```

4、Training

Copy the bash code

```
1  cd humanplus/scripts/utils/HIT
2  python3 imitate_episodes_h1_train.py --task_name data_cb_grasp --ckpt_dir
    cb_grasp/ --policy_class HIT --chunk_size 50 --hidden_dim 512 --batch_size 48
    --dim_feedforward 512 --lr 1e-5 --seed 0 --num_steps 100000 --eval_every 1000
    --validate_every 1000 --save_every 1000 --no_encoder --backbone resnet18 --
    same_backbones --use_pos_embd_image 1 --use_pos_embd_action 1 --dec_layers 6 -
    -gpu_id 0 --feature_loss_weight 0.005 --use_mask --data_aug
```

5、Reproduce/Evaluate

Copy the bash code

```
1  cd humanplus/scripts
2  python3 cb.py
```

1002- The Unidexgrasp dexterous hand grasping algorithm based on LinkerHand

The original Unidexgrasp algorithm uses the ShadowHand. The following provides the relevant code for developing the Unidexgrasp algorithm on LinkerHand.

[For detailed usage instructions, please refer to the linker_unidexgrasp](#)

Part of Generating the Grasping Posture

For the part of the grasping posture, a mapping scheme is adopted. The hand posture of the shadowhand output by the model is mapped to the hand posture of the LinkerHand L20 for subsequent development.

1. Configure the environment

Copy the code in the command line

```
1  conda create -n unidexgrasp python=3.8
2  conda activate unidexgrasp
3  conda install -y pytorch==1.10.0 torchvision==0.11.0 torchaudio==0.10.0
   cudatoolkit=11.3 -c pytorch -c conda-forge
4  conda install -y https://mirrors.bfsu.edu.cn/anaconda/cloud/pytorch3d/linux-
   64/pytorch3d-0.6.2-py38_cu113_pyt1100.tar.bz2
5  pip install -r requirements.txt
6  cd thirdparty/pytorch_kinematics
7  pip install -e .
8  cd ../nflows
9  pip install -e .
10 cd ../
11 git clone https://github.com/wrc042/CSDF.git
12 cd CSDF
13 pip install -e .
14 cd ../../
```

2. Training

GraspIPDF

Copy the code in the command line

```
1 python ./network/train.py --config-name ipdf_config \  
2 --exp-dir ./ipdf_train
```

GraspGlow

Copy the code in the command line

```
1 python ./network/train.py --config-name glow_config \  
2 --exp-dir ./glow_train  
3 python ./network/train.py --config-name glow_joint_config \  
4 --exp-dir ./glow_train
```

ContactNet

Copy the code in the command line

```
1 python ./network/train.py --config-name cm_net_config \  
2 --exp-dir ./cm_net_train
```

3. Verification

Copy the code in the command line

```
1 python ./network/eval.py --config-name eval_config \  
2 --exp-dir=./eval
```

1. Mapping

Visualize the results

Copy the code in the command line

```
1 python ./tests/visualize_result_l20_shadow.py --exp_dir 'eval' --num 3
```

Save the results for the subsequent development of the reinforcement learning algorithm.

Copy the code in the command line

```
1 python ./tests/data_for_RL.py
```




Linker Hand L20

产品手册



灵心巧手（北京）科技股份有限公司

版本修订记录

版本号	变更日期	变更说明
V1.0	2025.09.11	初版
V1.1	2025.11.24	更新参数及上位机说明
V1.2	2026.03.24	更新参数及上位机说明
V1.3	2026.08.04	更新关节角度值

安全须知

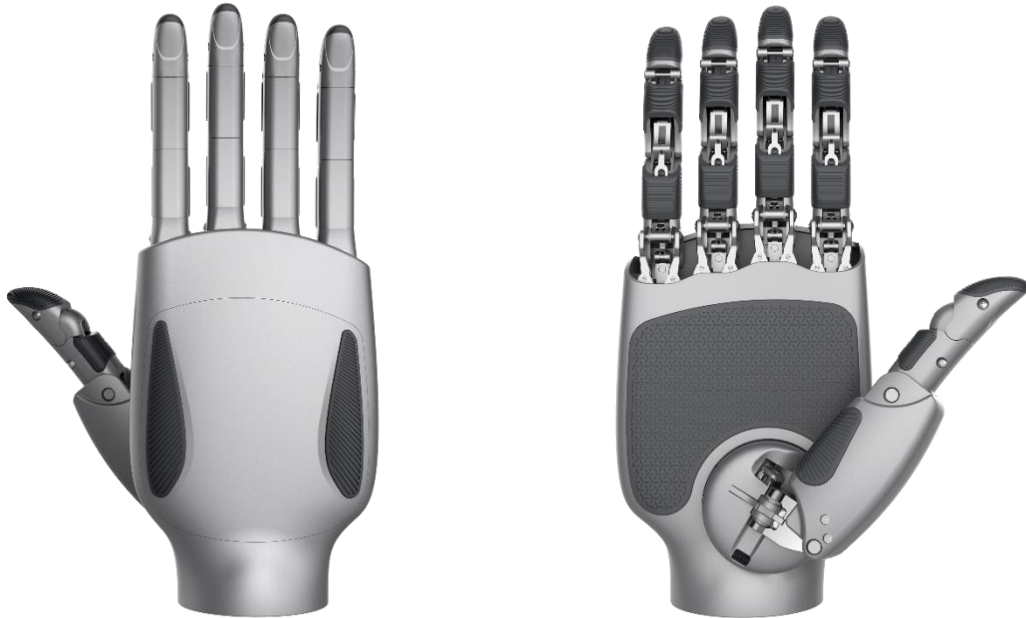
在安装或使用本产品前，请仔细阅读本手册和相关手册，手册中描述得相关安全事项，请务必仔细阅读相关说明。

1. 需确保本产品使用配套或规定的电缆进行连接，并按规定正确固定本产品，勿使用损坏的电源线、插头或松动的插座。
2. 本产品必须符合本手册描述的安裝要求，在超出所规定的安裝要求条件下使用本产品可能导致产品寿命缩短，并可能造成严重安全问题。
3. 本产品如长时间大功率运行，会出现温度升高的情况，当温度过高，请停止运行，静置冷却一段时间后进行使用。
4. 该设备只能由经过培训的工作人员使用，请勿私自打开外壳以及拆卸。如果设备出现故障，勿自行维修，请与我司售后服务部门联系。
5. 本手册中的图及照片为代表性示例，与所购买产品或有细节差异；另本手册会根据产品改进、规格变更等原因进行适当的修改。

目录

1	产品介绍	1
1.1	产品功能	2
1.2	外观尺寸	3
1.3	自由度与运动范围	3
1.5	产品参数	5
1.6	传感器系统	6
2	安装与调试	6
2.1	配件清单	7
2.2	安装说明	8
2.3	软件调试介绍	9
3	售后与服务须知	9

1 产品介绍



Linker Hand L20 产品图

Linker Hand L20 是 21 关节灵巧手，融合高性能无刷电机与精密滚珠丝杠，可模拟人手完成精准捏取、稳定抓取，兼具高功率密度、高效传动特性，能够为工业场景提供优于传统抓取器的方案。

该灵巧手性能上实现了高精度动作控制与稳定运动，可适配汽车高精度安装、航空航天特种作业、物流异形包裹分拣、高危环境替代作业等需求，是提升自动化水平的革命性工具。

1) 高精度控制与仿生操作

结合无刷电机与滚珠丝杠的微米级传动，实现了对位置和力矩的精确闭环控制，可胜任精密装配、易碎品抓取等精细作业。

2) 强劲高效的动力输出

无刷电机提供超高功率密度，结合高刚性连杆与滚珠丝杠的高效传动，额定负载达 20kg，开合时间仅需 0.9 秒，保障高效稳定的“抓-搬-放”作业流程。

3) 节能可靠的持续作业

额定静态电流仅 0.3A，功耗控制优异，支持 24V 工业电源及 24 小时不间断运行，满足高强度产线需求。

4) 轻量化与快速集成

整体轻量化设计，可快速安装部署，广泛适用于 3C、汽车、航空航天等高精度制造场景。

1.1 产品功能

1.1.1 位置控制

能够精准控制手部整体及各手指关节的空间位置，可平稳执行预设轨迹运动，确保从整体动作到单关节微调的操作准确性，满足精密装配、路径化作业等对位置精度要求较高的场景。

1.1.2 速度控制

可根据任务需求灵活调节手指运动速度，精准适配不同作业节奏—高速模式提升效率，低速模式保障精细操作安全，从而高效兼顾作业效率与操作安全性，适配多样化任务场景。

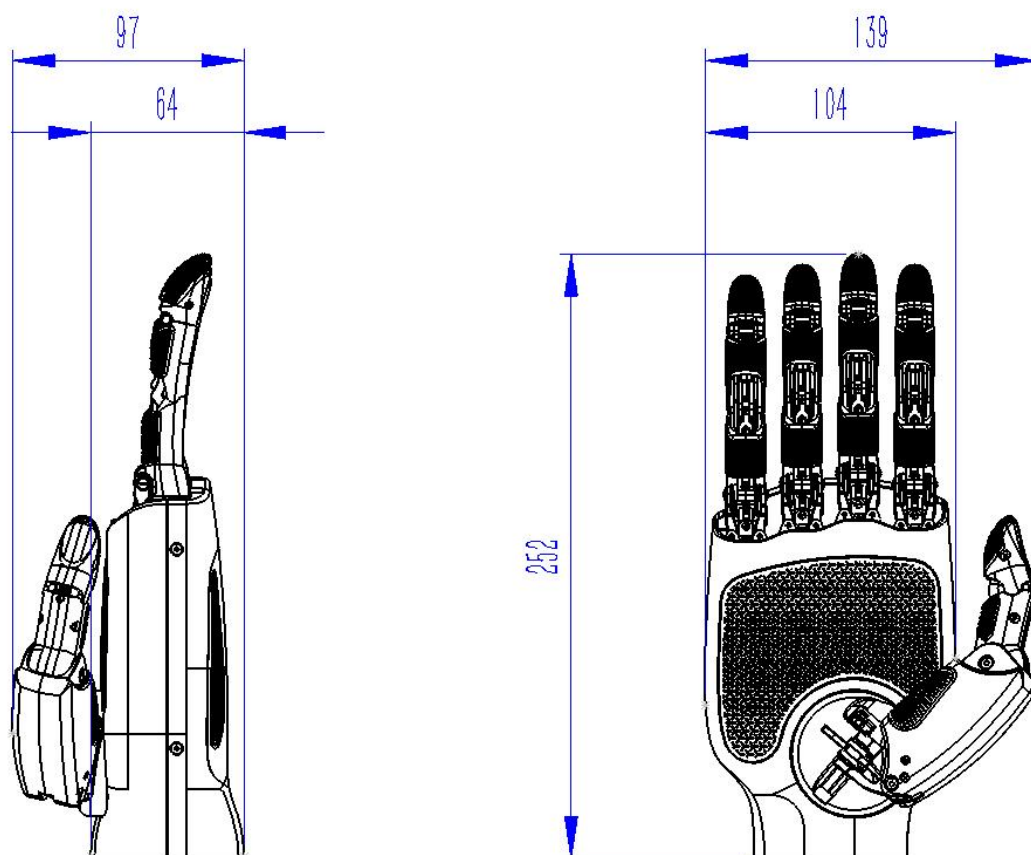
1.1.3 触觉反馈（力控）

借助指尖传感器实时感知并精准控制手指施加的力与扭矩大小，可动态调整输出力度：既避免因力度过大损伤脆弱物体，也防止力度不足导致物体滑落，为螺丝锁附、薄片抓取等精细操作提供可靠力控保障。

1.1.4 在线升级

支持通过上位机对灵巧手系统固件进行在线更新，可持续迭代功能模块、优化核心性能参数，适配需要长期升级迭代的工业场景与科研需求，保障设备长期处于高效运行状态。

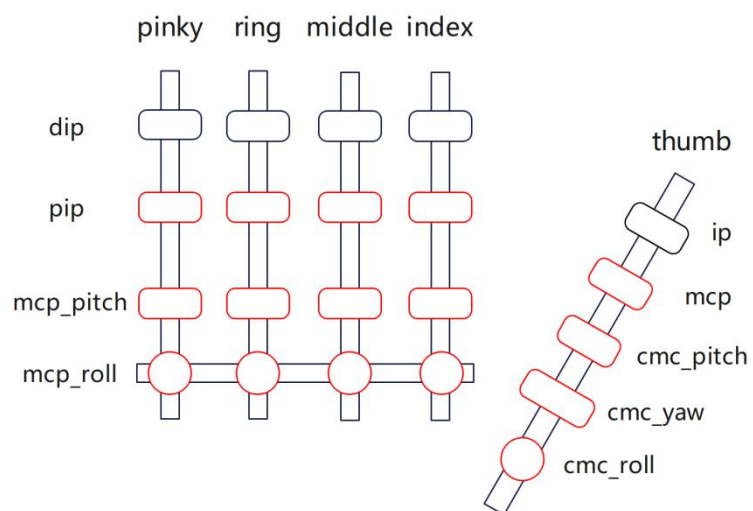
1.2 外观尺寸



外观尺寸图(单位 mm)

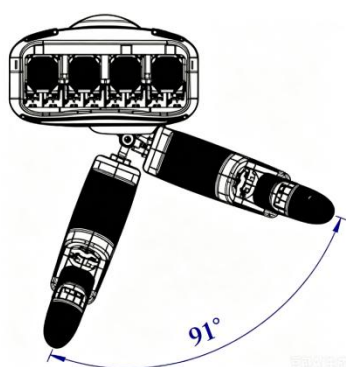
1.3 自由度与运动范围

本产品共 16 个自由度，21 个关节，其中 16 个主动关节，5 个被动关节，具体如下图。

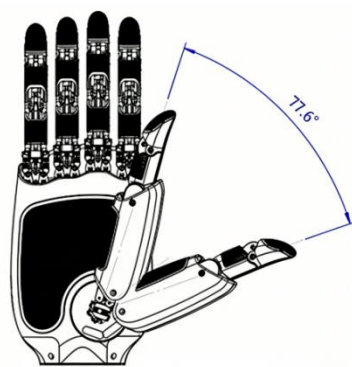


关节自由度图

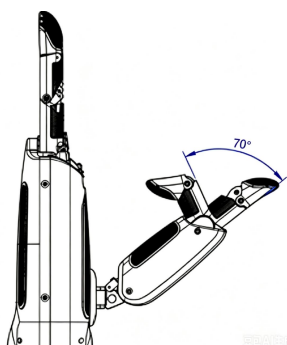
以下为本产品各关节的实际活动范围。



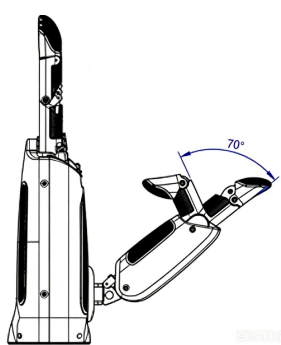
Thumb cmc_yaw



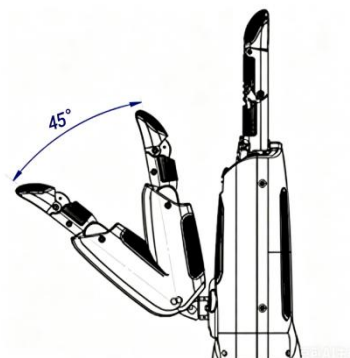
Thumb cmc_roll



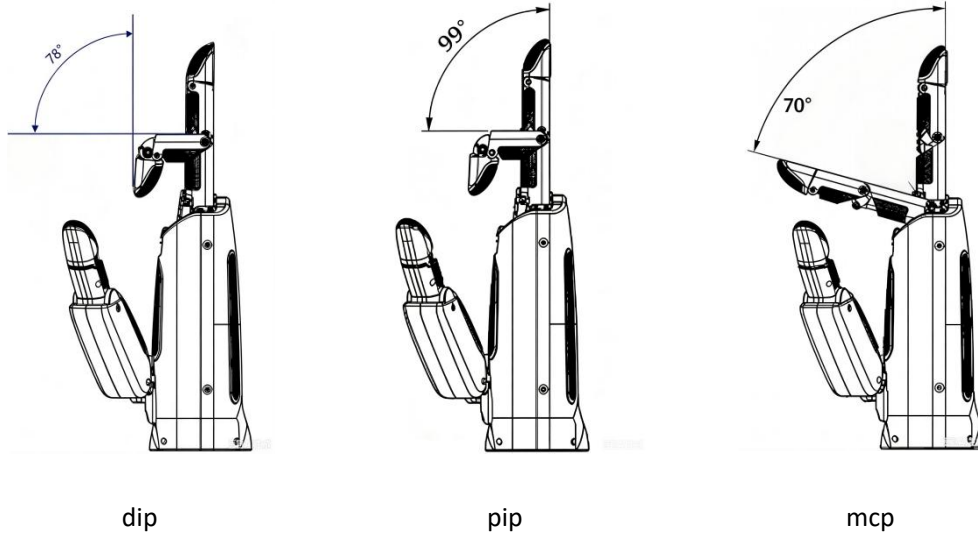
ip



mcp



cmc_pitch



下表为本产品各关节的实际活动范围与活动速度。

关节(Joints)	最大运动角度 (°)	最大运动速度 (° /s)
ip	71	133.96
mcp	70	132.08
cmc_pitch	45	80.36
cmc_yaw	91	185.71
cmc_roll	77.6	129
dip	78	195
pip	99	247.5
mcp_pitch	70	194.44
mcp_roll	±13	68.42

1.4 产品参数

1.4.1 基础参数

型号	Linker hand L20
自由度	16
关节数	21 (16 主动 + 5 被动)
传动方式	连杆传动

控制接口	CAN/RS485
通信频率	500Hz
重量	≈1200g
最大负载	20kg
工作电压	DC 24V—48V
静态电流	0.3A
空载运动平均电流	0.4A
最大电流	1.8A
重复定位精度	±0.2mm
开合时间	0.98s

1.4.2 力性能参数

性能指标	具体参数
拇指最大指尖力	15N
四指最大指尖力	20N
五指最大抓握力	100N

1.5 传感器系统

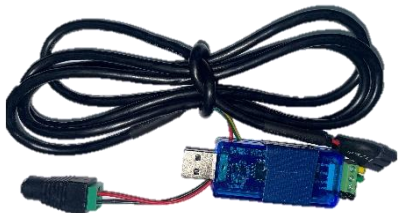
Linker Hand L20 配备压敏传感器，具体参数如下。

参数	规格说明
压阻阵列	6*12
传感器受力区域	9.6*14.4mm
触发力	5g
量程	20N
寿命	10 万次
通信帧率	200FPS

2 安装与调试

2.1 配件清单

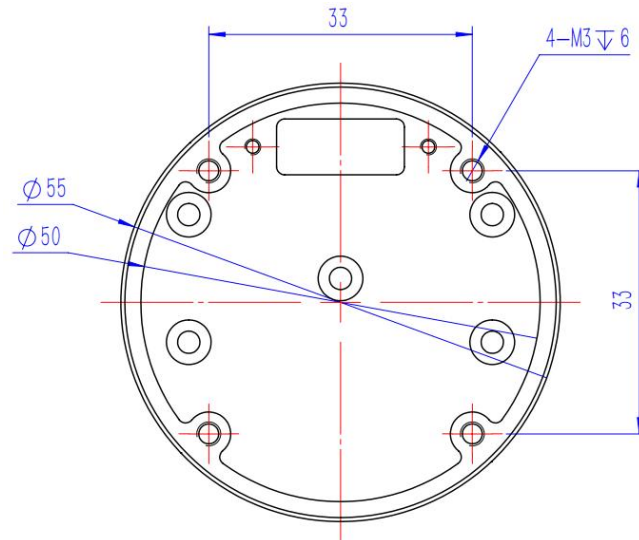
安装灵巧手前，先检查配件是否齐全。

序号	名称	数量	图片
1	USB 转 CAN 调试线	1	
2	接插线 XT30 (2+2)	1	
3	电源适配器	1	
4	电源线	1	

2.2 安装说明

2.2.1 结构安装说明

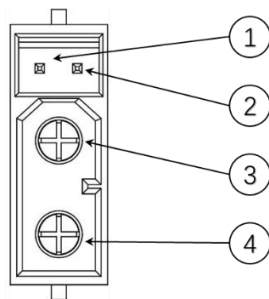
安装需使用 M3 螺丝，具体长度根据固定件厚度调整，安装法兰结构图如下所示。



法兰结构图

2.2.2 电气安装说明

灵巧手采用型号为 XT30 (2+2) 接插件，其线序如下表。



线序	说明
1	CAN L
2	CAN H
3	GND
4	24V

2.2 调试线连接

将使用 USB 转 CAN 调试线将灵巧手通过 USB 线与调试 PC 连接, 将电源线与电源适配器插好后, 并插到 USB 转 CAN 调试线的电源口上, 另一端插到 AC220V 电源上。

3 售后与服务须知

- 1) 产品享受自购买之日起 12 个月的有限保修。
- 2) 保修期内, 因制造或材料缺陷导致的故障, 我们将提供免费维修或部件更换。
- 3) 免责条款: 以下情况不属保修范围:
 - 正常使用造成的外观磨损。
 - 因不当操作、意外、私自拆卸或未遵循指南导致的损坏。
- 4) 非质量问题, 不予退换。
- 5) 产品出现任何故障, 请立即联系官方售后, 切勿自行拆卸。
- 6) 保修期外的维修将收取相应费用。

附录

ROS SDK 获取路径: <https://github.com/linker-bot/linkerhand-ros-sdk>

ROS2 SDK 获取路径: <https://github.com/linker-bot/linkerhand-ros2-sdk>

Python SDK <https://github.com/linker-bot/linkerhand-python-sdk>

技术支持邮箱: support@linkerbot.cn

如需更多产品资料, 可访问公司官网: <https://linkerbot.cn>

灵心巧手（北京）科技股份有限公司

联系方式: 010-53607783; +86 13911526671

商务邮箱: business@linkerbot.cn

人才招聘: hr@linkerbot.cn

公司地址: 北京市海淀区大钟寺东路 168 号

